Lab: Ontology and Ontology Development in OWL

Submitted by:

Lab Group 39
Ardra Suresh
Reshma Mani

1. Ontology Definition

Ontology is defined as “An ontology is an explicit, formal specification of a shared conceptualization. It can be broken down into its key components:

- **Explicit:** The conceptualization is clearly and overtly stated, not implicit or hidden
- **Formal:** It follows a rigorous, structured approach that can be processed computationally
- **Specification:** It provides a detailed description of the elements and their relationships
- **Shared:** It represents a consensus view agreed upon by a community, not just an individual perspective
- **Conceptualization**: It captures an abstract model of how people think about things in a particular domain

This definition originated from Tom Gruber (1993), an American computer scientist who defined ontology in AI terms: “An ontology is an explicit specification of a conceptualization.” And pioneered knowledge sharing and interoperability. So it is also presents an equivalent technical definition: "A description logic knowledge base," which connects ontologies to formal logic systems.

2. Web Ontology Language (OWL)

OWL 2 DL

- Is an extension of SROIQ (a description logic)
- Is decidable - meaning reasoning problems are guaranteed to terminate with an answer
- Has several production quality reasons that cover the entire language: FaCT++, HermiT, Pellet, and RacerPro
- Based on direct semantics
- Provides practical computational reasoning capabilities

OWL 2 Full

- Based on RDF-compatible semantics
- Is undecidable - reasoning problems may not terminate
- Has no reasoned available for OWL 2 Full
- Provides very high expressiveness and all syntactic freedom of RDF (a graph-oriented formalism)
- Offers more flexibility but sacrifices computational guarantees

Three Profiles of OWL 2 DL

OWL 2 EL

- Particularly suitable for applications employing ontologies that define very large numbers of classes and/or properties
- Optimized for scale in terms of vocabulary size

OWL 2 QL

- Aimed at applications that use very large volumes of instance data
- Query answering is the most important reasoning task

OWL 2 RL

- Aimed at applications that require scalable reasoning without sacrificing too much expressive power
- Suitable for rule-based reasoning approaches

3. Using Prot´eg´e

Downloaded Prot´eg´e.

4. The Family Ontology

1) Find the classes HappyPerson, JohnsChildren, NarcisticPerson, and Orphan in the ontology, give the descriptions of the classes using DL syntax and natural language

a) HappyPerson :
DL syntax : HappyPerson ≡ (∃ hasChild.Happy) ⊓ (∀ hasChild.Happy)
Natural Language : A HappyPerson is someone who has at least one Happy child and all of their children are Happy.

b) JohnsChildren :
DL syntax : JohnsChildren ≡ ∀ hasParent.{John}
Natural Language : JohnsChildren are persons whose parent is John.

c) NarcisticPerson :
DL syntax : NarcisticPerson ≡ ∃ loves.Self
Natural Language : Someone who loves themselves.

d) Orphan :
DL syntax : Orphan ≡ ∀ $(\text{hasChild})^{-}$. Dead
Natural Language : A person whose parents are all dead.

2) Find the object properties hasSpouse and hasWife, give the descriptions and/or characteristics of the object properties using natural language.

a)hasSpouse

- ObjectProperty: hasSpouse, Characteristics: Symmetric"
- Natural Language Description: The hasSpouse property is symmetric, meaning if person A has person B as a spouse, then person B automatically has person A as a spouse. The relationship is bidirectional and mutual.

b)hasWife

- hasWife Domain: Man" means ∃hasWife.⊤ ⊑ Man (only men can have wives)
- Range: Woman - "hasWife Range: Woman" means ∀hasWife.Woman ⊑ ⊤ (all wives must be women)
- ObjectProperty: hasHusband, Characteristics: Functional
- Characteristic: Asymmetric
- Inverse Property: hasHusband
- Natural Language Description: The hasWife property relates a man to a woman. Being functional means that each man can have at most one wife (if a man has multiple hasWife assertions, they must refer to the same individual). The property is asymmetric, so if a man has a wife, that woman cannot have that man as a wife. It is the inverse of hasHusband.

3) Find the class Teenager in the ontology, give the description of the class in natural language, and the description and characteristics of the datatype property hasAge.

a)Teenager

- DL Syntax: Teenager ⊑ Person ⊓ ∃hasAge.integer[≥ 12, < 19]
- Natural Language: A teenager is defined as a person whose age is an integer value greater than or equal to 12 and less than 19. The lecture shows this exact example: "Class: Teenager, SubClassOf: hasAge some integer[< 12, >= 19]"

b)hasAge

- Type: Datatype Property
- Domain: Person - relates persons to their age values
- Range: xsd:nonNegativeInteger

4) Find the individual John, give its description using natural language

Natural Language Description of John: John is an individual in the family ontology with the following characteristics:
Father :

hasChild exactly 3 Parent : This person has exactly three children, and all of them are Parents.

hasChild exactly 5 owl.thing : This person has exactly five children, of any type.

hasChild max 4 Parent : This person has at most 4 children, and all of them are parents.

hasChild min 2 Parent : This person has at least 2 children, and all of them are parents.

Different Individuals- Bill : John is a different person from Bill

hasWife Mary : This person has a wife Mary

has Age 51 : This person’s age is 51

5) Choose the HermiT reasoner from the Reasoner menu and click Start Reasoner. Then open the DL Query tab (from Window →Tabs menu) and set Query for to includeSuperclasses, Subclasses, Instances. Copy each of the following DL queries (in the Manchester syntax) into the Query box, click Execute, and write down the answers returned by the reasoner. For each result, explain why it was generated by the reasoningprocess.
• Grandfather
• ChildlessPerson
• Person and (hasChild min 2 Person)

Grandfather : Man and (hasChild some (hasChild some Person))
Reasoners : John
Explanation : The class Grandfather is defined in the ontology as a Man who has a child and that also has a child. John is male and have children who themselves have children.

ChildlessPerson : Person and (hasChild max 0 Person)
Reasoners : Jack
Explanation : The ChildlessPerson is defined as a person who has no child. Jack is a person and who has no child.

Person and (hasChild min 2 Person)
Reasoners : John
Explanation : This is a person and who has at least 2 children.

6) Go back to the Entities and browse through the classes, properties, and individuals in the ontology. Observe the changes (highlighted in yellow) introduced by the reasoner. You may notice that the query results above correspond to some of these inferred changes. If you click the quesiton mark icon, you can see the explaination provided for the reasoning results. Choose one of the inferred changes for anobject property andan individual, and explain why it was inferred by the reasoner.

Classes : JohnsChildren

Explanation : JohnsChildren EquivalentTo hasParent value John. An individual belongs to JohnsChildren if and only if their parent is John.

5. An Ontology for Educational Resources in European Universities

1) Purpose and Scope

The purpose of this ontology is to represent conceptual knowledge about universities, their educational programs, course structures, and academic participants such as professors and students.

This ontology models relationships such as:

- which universities offer which programs,
- which programs include which courses,
- which student takes which course,
- which country a university belongs to.

In scope:

- University → Program → Course structure
- Types of programs
- Course participation (students)
- Course teaching (professors)
- Geographic relations (universities and countries)

2) Requirements via Competency Questions (CQs)

1. Which programs are offered by a given university?
2. Which courses belong to a selected academic program?
3. Which students are taking a specific course?
4. Which universities are located in a particular country?
5. What courses does a specific student take?
6. Which universities offer programs that include Machine Learning or related courses?

3) Conceptualization and Modeling

Key Terms

- **University**
- **Program**
- **Course**
- **Country**
- **Person**
  - Student
  - Professor
- **Academic Activities**
  - Teaching
  - Enrollment
  - Course belonging

3.1 Classes of the Ontology

Core Classes

- University
- Program
- Course
- Person
- Country

Person subclasses

- Professor
- Student

4) Object and Datatype Properties

4.1 Object Properties

Property	Domain	Range	Description
offerprogram	University	Program	A university offers a program
hascorse	Program	Course	A program includes a course
taughtby	Course	Professor	A course is taught by a professor
Takescourse	Student	Course	A student takes a course
locatedIn	University	country	A university is located in a country

5) Axioms and Restrictions

5.1 Class Description Axioms

- University ⊑ locatedIn some Country
- Program ⊑ offerProgram inverse some University
- Course ⊑ taughtBy some Professor

5.2 Property Restrictions

- Course ⊑ (taughtby only Professor)
- Student ⊑ (Takescourse only Course)
- Program ⊑ (hascourse some Course)
- University ⊑ (offerprogram some Program)

5.3 Property Characteristics

- **locatedIn** → Functional (a university belongs to exactly one country)
- **Taughtby** → Functional

6) Individuals (Instances)

Below are **instances based on your Protégé model**.

University

Individual: jonkopinuniversity

- Type: **University**
- Facts:
  - locatedIn: Sweden
  - offerProgram: AI_master

Country

Individual: Sweden

- Type: **Country**
- Facts:
  - hasName: “Sweden”

Programs

Individual: AI_Master

  - hasCourse: Machinelearning

Student

Individual: Jenny

- Type: **Student**

  Facts:

- Takecourse: Machinelearning